Project Development Report

Chemical Equipment Parameter Visualizer

Generated on: 2026-02-02 19:09

1. Project Overview

The goal was to build a functional **Hybrid Application** (Web + Desktop) for visualizing chemical equipment data. The system consists of a centralized Django backend that serves two different frontends: a React-based web app and a PyQt5-based desktop app.

2. Backend Architecture (Django)

- **REST API**: Built using Django REST Framework to handle CSV uploads, parsing, and data retrieval.
- **Data Processing**: Integrated *Pandas* to read CSVs, calculate flowrate/pressure averages, and analyze equipment type distributions.
- **PDF Generation**: Implemented an endpoint using *ReportLab* to generate dynamic PDF reports of equipment data.
- **Authentication**: Secured endpoints using Basic Authentication and Django's User system.
- **Cloud Readiness**: Configured *WhiteNoise* for static files and *Gunicorn* for production serving.

3. Web Frontend (React + Vite)

- **Tech Stack**: React.js, Chart.js, Axios, Vite.
- **UI Design**: Implemented a 'Premium' aesthetic using CSS variables, gradients, glassmorphism effects, and the 'Inter' font.
- **Features**: Login screen, File Upload with progress, History sidebar, and Interactive Charts.
- **Deployment**: Configured for Vercel deployment, ensuring proper handling of environment variables like *VITE_API_URL* for connecting to the production backend.

4. Desktop Frontend (PyQt5)

- **GUI Implementation**: Built a native Windows interface using PyQt5 widgets (*QTabWidget, QTableWidget*).
- **Visualization**: Embedded *Matplotlib* figures directly into the PyQt interface for data plotting.
- **Integration**: Created a custom *APIClient* with session management to handle authenticated requests to the Django backend.
- **Stability**: Fixed crashing issues by implementing proper threading and error handling during login.

5. Deployment & Challenges Solved

- **Cross-Platform Git Issues**: Solved a critical issue where Windows *node_modules* were accidentally pushed to GitHub, breaking Linux builds. Fixed by updating *.gitignore* and clearing the git cache.
- **Backend Hosting**: Deployed Django to **Render** using a custom *Procfile* and environment variables for security.
- **Frontend Hosting**: Deployed React to **Vercel**, configuring it to communicate with the Render backend via secure HTTPS.

Conclusion

The project is now fully functional and deployed. Users can upload data via the Web or Desktop, and the persistent SQLite database syncs the history across both platforms.